

Kata Bank Account con TDD en Java

Gabriel Alexandru Iacob

2º DAM - IES Francisco de Goya

Entregable: capturas de pantalla del código y los tests.

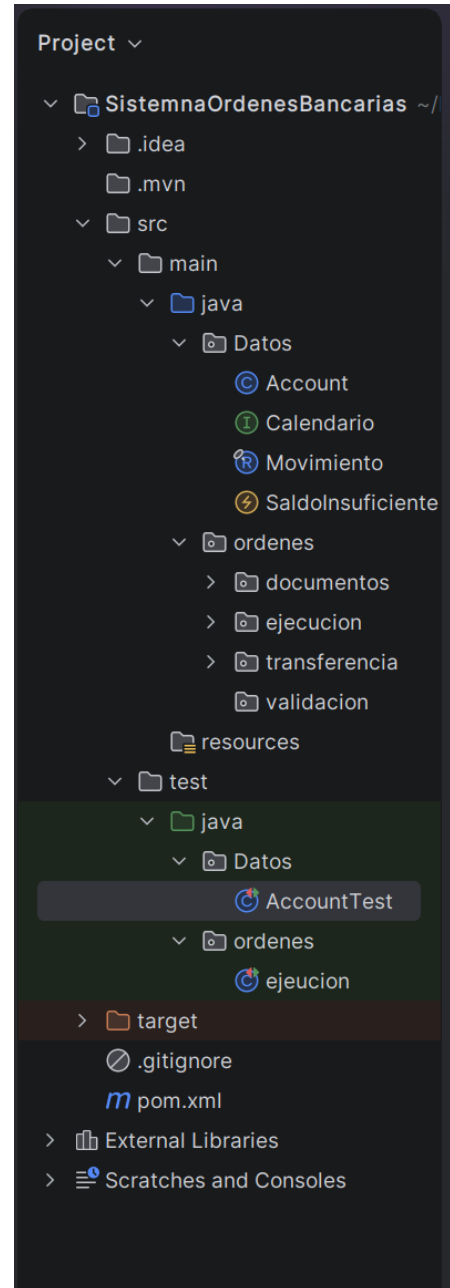
Requisitos: clase Account con deposit(int), withdraw(int) y printStatement().

Formato del extracto: fecha d.M.yyyy, importe con signo y saldo acumulado.

Date	Amount	Balance
24.12.2015	+500	500
23.8.2016	-100	400

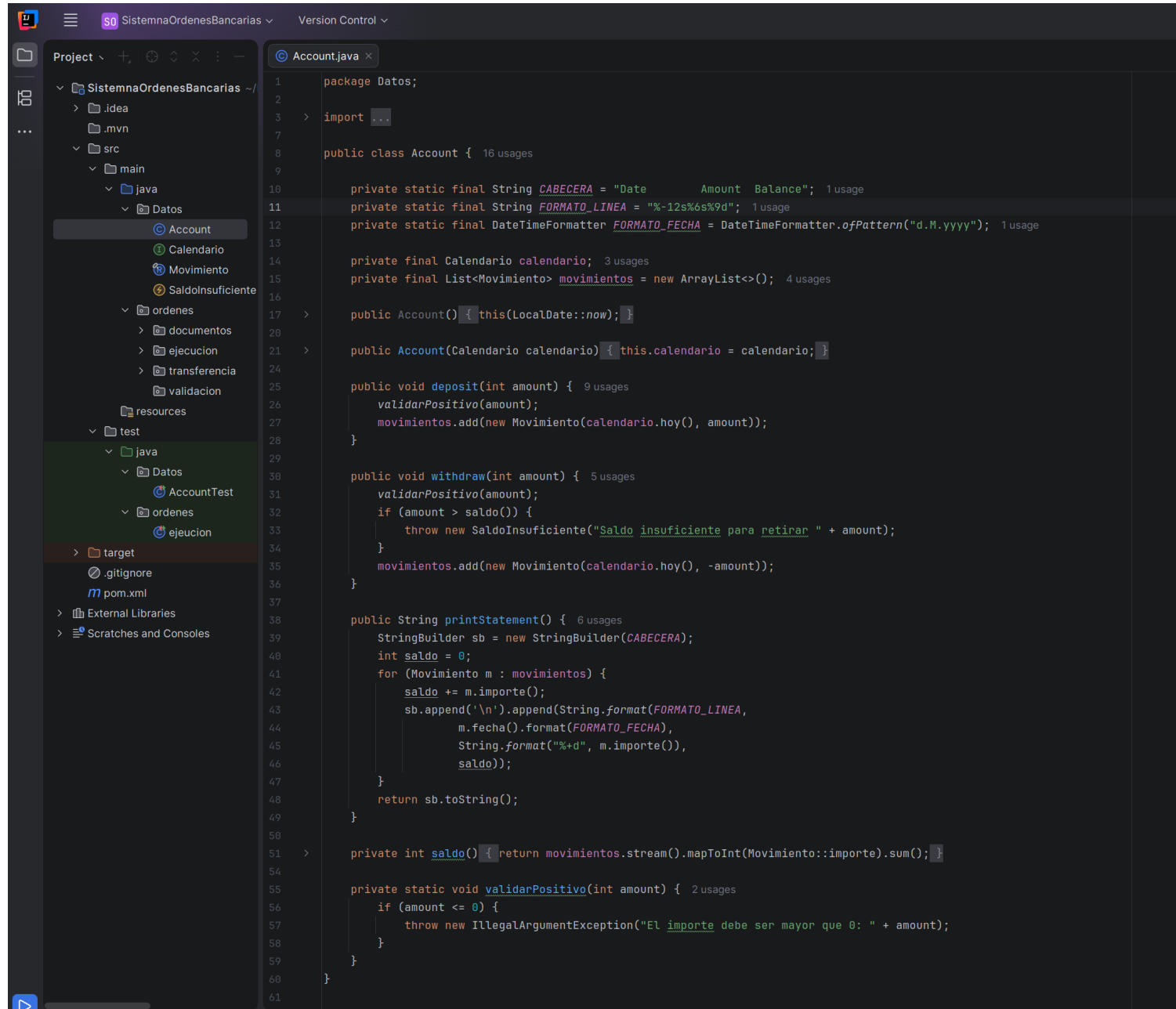
1. Estructura del proyecto

Clases de producción en `src/main/java/Datos` y tests en `src/test/java/Datos`.



2. Account.java

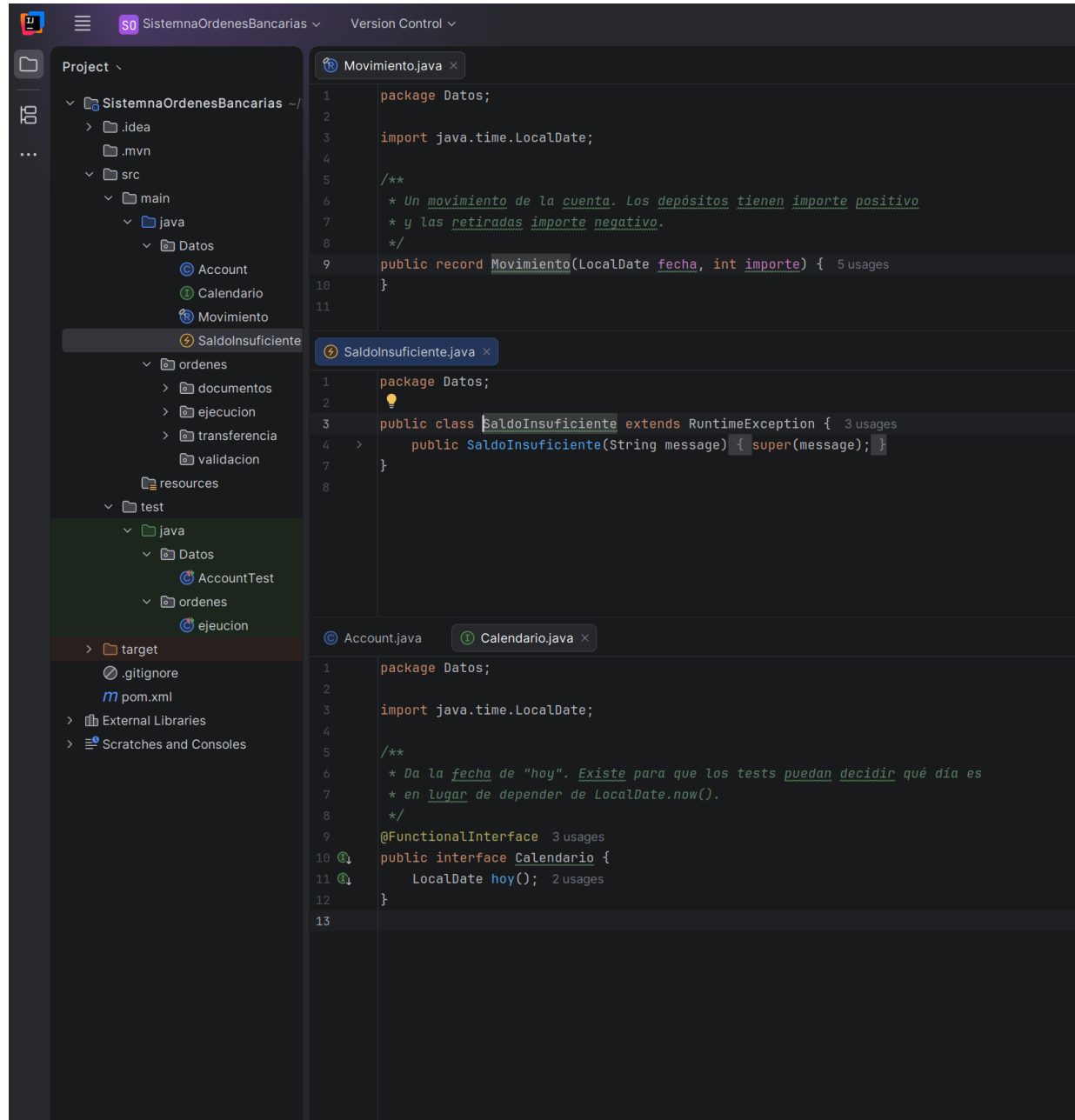
Clase principal: deposit, withdraw y printStatement. La fecha se obtiene de un Calendario inyectado.



```
1 package Datos;
2
3 import ...
4
5 public class Account { 16 usages
6
7     private static final String CABECERA = "Date      Amount  Balance"; 1 usage
8     private static final String FORMATO_LINEA = "%-12s%6s%9d"; 1 usage
9     private static final DateTimeFormatter FORMATO_FECHA = DateTimeFormatter.ofPattern("d.M.yyyy"); 1 usage
10
11     private final Calendario calendario; 3 usages
12     private final List<Movimiento> movimientos = new ArrayList<>(); 4 usages
13
14     public Account() { this(LocalDate.now()); }
15
16     public Account(Calendario calendario) { this.calendario = calendario; }
17
18     public void deposit(int amount) { 9 usages
19         validarPositivo(amount);
20         movimientos.add(new Movimiento(calendario.hoy(), amount));
21     }
22
23     public void withdraw(int amount) { 5 usages
24         validarPositivo(amount);
25         if (amount > saldo()) {
26             throw new SaldoInsuficiente("Saldo insuficiente para retirar " + amount);
27         }
28         movimientos.add(new Movimiento(calendario.hoy(), -amount));
29     }
30
31     public String printStatement() { 6 usages
32         StringBuilder sb = new StringBuilder(CABECERA);
33         int saldo = 0;
34         for (Movimiento m : movimientos) {
35             saldo += m.importe();
36             sb.append('\n').append(String.format(FORMATO_LINEA,
37                 m.fecha().format(FORMATO_FECHA),
38                 String.format("%+d", m.importe()),
39                 saldo));
40         }
41         return sb.toString();
42     }
43
44     private int saldo() { return movimientos.stream().mapToInt(Movimiento::importe).sum(); }
45
46     private static void validarPositivo(int amount) { 2 usages
47         if (amount <= 0) {
48             throw new IllegalArgumentException("El importe debe ser mayor que 0: " + amount);
49         }
50     }
51 }
52
53
54
55
56
57
58
59
60
61
```

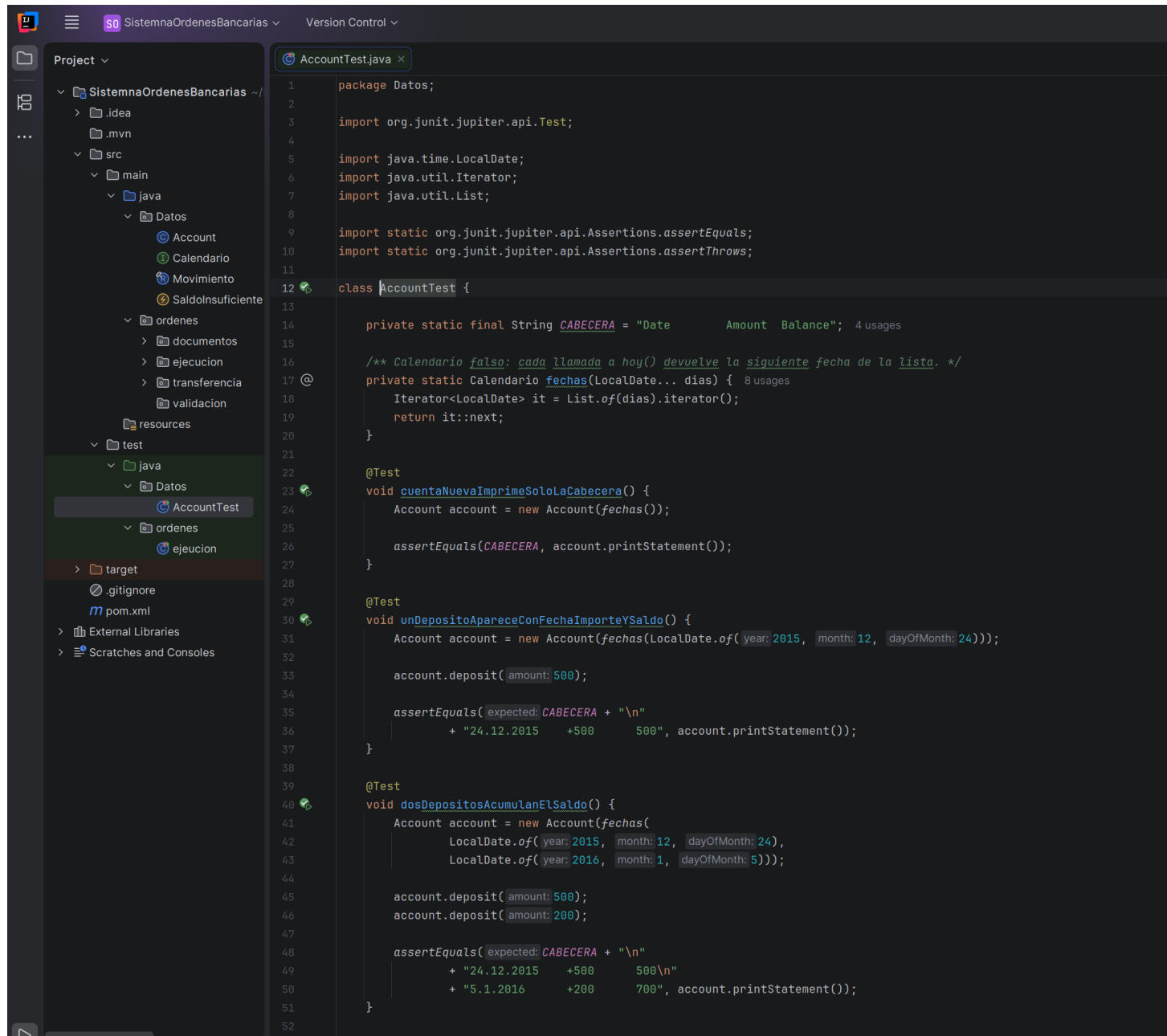
3. Movimiento, SaldoInsuficiente y Calendario

Record de movimiento, excepción de saldo insuficiente e interfaz que permite controlar la fecha en los tests.



4. AccountTest.java (1/2)

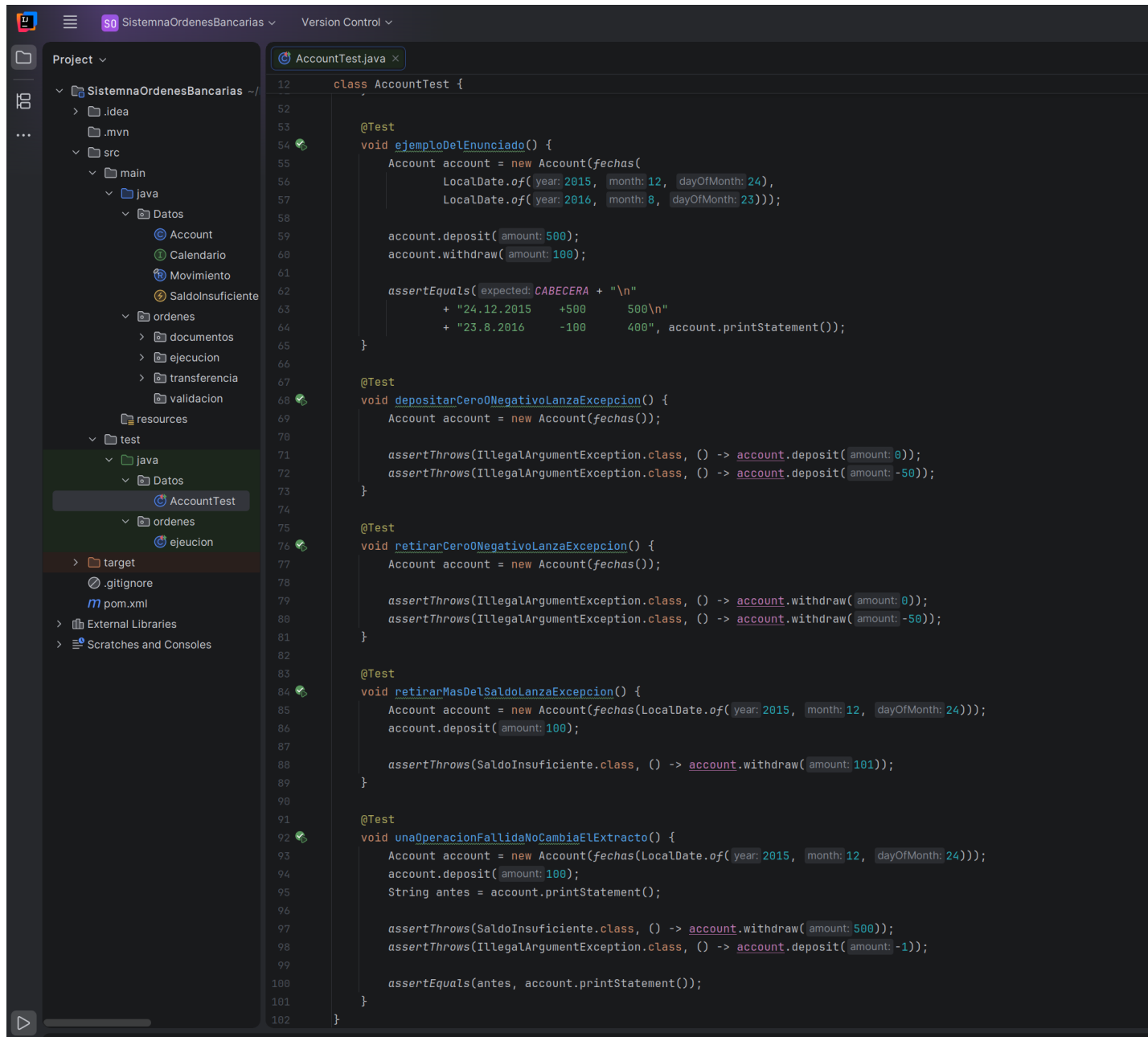
Calendario falso para los tests, cabecera, un depósito y dos depósitos con saldo acumulado.



```
1 package Datos;
2
3 import org.junit.jupiter.api.Test;
4
5 import java.time.LocalDate;
6 import java.util.Iterator;
7 import java.util.List;
8
9 import static org.junit.jupiter.api.Assertions.assertEquals;
10 import static org.junit.jupiter.api.Assertions.assertThrows;
11
12 class AccountTest {
13
14     private static final String CABECERA = "Date      Amount  Balance";
15
16     /** Calendario falso: cada llamada a hoy() devuelve la siguiente fecha de la lista. */
17     private static Calendaro fechas(LocalDate... dias) {
18         Iterator<LocalDate> it = List.of(dias).iterator();
19         return it::next;
20     }
21
22     @Test
23     void cuentaNuevaImprimeSoloLaCabecera() {
24         Account account = new Account(fechas());
25
26         assertEquals(CABECERA, account.printStatement());
27     }
28
29     @Test
30     void unDepositoApareceConFechaImporteYSaldo() {
31         Account account = new Account(fechas(LocalDate.of(2015, 12, 24)));
32
33         account.deposit(500);
34
35         assertEquals("expected: CABECERA + "\n"
36             + "24.12.2015    +500    500", account.printStatement());
37     }
38
39     @Test
40     void dosDepositosAcumulanElSaldo() {
41         Account account = new Account(fechas(
42             LocalDate.of(2015, 12, 24),
43             LocalDate.of(2016, 1, 5)));
44
45         account.deposit(500);
46         account.deposit(200);
47
48         assertEquals("expected: CABECERA + "\n"
49             + "24.12.2015    +500    500\n"
50             + "5.1.2016      +200    700", account.printStatement());
51     }
52 }
```

5. AccountTest.java (2/2)

Ejemplo del enunciado, validación de importes, saldo insuficiente y extracto sin cambios tras un error.



```
12 class AccountTest {
13
52
53
54     @Test
55     void ejemploDelEnunciado() {
56         Account account = new Account(fechas(
57             LocalDate.of(year: 2015, month: 12, dayOfMonth: 24),
58             LocalDate.of(year: 2016, month: 8, dayOfMonth: 23)));
59
60         account.deposit(amount: 500);
61         account.withdraw(amount: 100);
62
63         assertEquals(expected: CABECERA + "\n"
64             + "24.12.2015 +500 500\n"
65             + "23.8.2016 -100 400", account.printStatement());
66     }
67
68     @Test
69     void depositarCeroONegativoLanzaExcepcion() {
70         Account account = new Account(fechas());
71
72         assertThrows(IllegalArgumentException.class, () -> account.deposit(amount: 0));
73         assertThrows(IllegalArgumentException.class, () -> account.deposit(amount: -50));
74     }
75
76     @Test
77     void retirarCeroONegativoLanzaExcepcion() {
78         Account account = new Account(fechas());
79
80         assertThrows(IllegalArgumentException.class, () -> account.withdraw(amount: 0));
81         assertThrows(IllegalArgumentException.class, () -> account.withdraw(amount: -50));
82     }
83
84     @Test
85     void retirarMasDelSaldoLanzaExcepcion() {
86         Account account = new Account(fechas(LocalDate.of(year: 2015, month: 12, dayOfMonth: 24)));
87         account.deposit(amount: 100);
88
89         assertThrows(SaldoInsuficiente.class, () -> account.withdraw(amount: 101));
90     }
91
92     @Test
93     void unaOperacionFallidaNoCambiaElExtracto() {
94         Account account = new Account(fechas(LocalDate.of(year: 2015, month: 12, dayOfMonth: 24)));
95         account.deposit(amount: 100);
96         String antes = account.printStatement();
97
98         assertThrows(SaldoInsuficiente.class, () -> account.withdraw(amount: 500));
99         assertThrows(IllegalArgumentException.class, () -> account.deposit(amount: -1));
100
101         assertEquals(antes, account.printStatement());
102     }
103 }
```

6. Resultado de los tests

Ejecución de AccountTest en IntelliJ: 8 tests superados.

